

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

**Oleh: Anggraeni Dwi Zahra
NIM. 2410817220018**

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : AnggraeniDwi Zahra
NIM : 2410817220018

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code	7
B. Output Program.....	37
C. Pembahasan.....	40
D. Tautan Git	46
SOAL 2	47
A. Pembahasan.....	47

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose (Step Into).....	37
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose (Step Over)	37
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose (Step Out)	38
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML (Step Into).....	38
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML (Step Over)	39
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML (Step Out)	39

DAFTAR TABEL

Tabel 1. Source Code MainActivity Compose Soal 1	7
Tabel 2. Source Code MainApplication Compose Soal 1.....	7
Tabel 3. Source Code Hotel Compose Soal 1	8
Tabel 4. Source Code HotelScreens Compose Soal 1	8
Tabel 5. Source Code HotelViewModel Compose Soal 1	16
Tabel 6. Source Code HotelViewModelFactory Compose Soal 1.....	18
Tabel 7. Source Code HotelFragment Compose Soal 1.....	18
Tabel 8. Source Code LanguageScreen Compose Soal 1	19
Tabel 9. Source Code LanguageViewModel Compose Soal 1	21
Tabel 10. Source Code Mainactivity XML Soal 1.....	22
Tabel 11. Source Code MainApplication XML Soal 1.....	22
Tabel 12. Source Code HotelViewModel XML Soal 1	23
Tabel 13. Source Code HotelViewModelFactory XML Soal 1.....	24
Tabel 14. Source Code HotelAdapter.kt XML Soal 1	24
Tabel 15. Source Code DetailActivity XML Soal 1	27
Tabel 16. Source Code activity_main XML Soal 1	28
Tabel 17. Source Code activity_detail. XML Soal 1	29
Tabel 18. Source Code item_hotel XML Soal 1	32
Tabel 19. Source Code fragment_home XML Soal 1.....	34

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

COMPOSE

MainActivity.kt

Tabel 1. Source Code MainActivity Compose Soal 1

```
1 package com.example.modul3compose
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.viewModels
7 import
8 com.example.modul3compose.ui.theme.Modul3ComposeTheme
9 import timber.log.Timber
10
11 class MainActivity : ComponentActivity() {
12     private val viewModel: HotelViewModel by viewModels {
13         HotelViewModelFactory("PROMO_PELAJAR")
14     }
15
16     override fun onCreate(savedInstanceState: Bundle?) {
17         super.onCreate(savedInstanceState)
18
19         setContent {
20             Modul3ComposeTheme {
21                 HotelApp(viewModel = viewModel)
22             }
23         }
24
25         Timber.d("DEBUG_LOG: MainActivity initialized
26 successfully with ComponentActivity")
27         Timber.i("PROMO_CODE: Promo PELAJAR has been
28 applied to ViewModel")
29     }
30 }
```

MainApplication.kt

Tabel 2. Source Code MainApplication Compose Soal 1

```
1 package com.example.modul3compose
2
3 import android.app.Application
4 import timber.log.Timber
5
6 class MyApplication : Application() {
```

7	override fun onCreate() {
8	super.onCreate()
9	if
	(android.content.pm.ApplicationInfo.FLAG_DEBUGGABLE !=
	0) {
10	Timber.plant(Timber.DebugTree())
11	}
12	}
13	}

Hotel.kt

Tabel 3. Source Code Hotel Compose Soal 1

1	package com.example.modul3compose
2	
3	data class Hotel (
4	val name: String,
5	val stars: String,
6	val location: String,
7	val price: String,
8	val imageURL: Int
9	)

HotelScreens.kt

Tabel 4. Source Code HotelScreens Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.clickable
7	import androidx.compose.foundation.layout.Arrangement
8	import androidx.compose.foundation.layout.Box
9	import androidx.compose.foundation.layout.Column
10	import androidx.compose.foundation.layout.PaddingValues
11	import androidx.compose.foundation.layout.Row
12	import androidx.compose.foundation.layout.Spacer
13	import androidx.compose.foundation.layout.fillMaxSize
14	import androidx.compose.foundation.layout.fillMaxWidth
15	import androidx.compose.foundation.layout.height
16	import androidx.compose.foundation.layout.padding
17	import androidx.compose.foundation.layout.size
18	import androidx.compose.foundation.layout.width
19	import androidx.compose.foundation.lazy.LazyColumn
20	import androidx.compose.foundation.lazy.LazyRow
21	import androidx.compose.foundation.lazy.items

```

22 import androidx.compose.foundation.rememberScrollState
23 import
   androidx.compose.foundation.shape.RoundedCornerShape
24 import androidx.compose.foundation.verticalScroll
25 import androidx.compose.material.icons.Icons
26 import androidx.compose.material.icons.filled.Language
27 import androidx.compose.material3.Button
28 import androidx.compose.material3.Card
29 import androidx.compose.material3.CardDefaults
30 import androidx.compose.material3.Icon
31 import androidx.compose.material3.IconButton
32 import androidx.compose.material3.MaterialTheme
33 import androidx.compose.material3.Text
34 import androidx.compose.runtime.Composable
35 import androidx.compose.runtime.collectAsState
36 import androidx.compose.runtime.getValue
37 import androidx.compose.ui.Alignment
38 import androidx.compose.ui.Modifier
39 import androidx.compose.ui.draw.clip
40 import androidx.compose.ui.graphics.Color
41 import androidx.compose.ui.layout.ContentScale
42 import androidx.compose.ui.platform.LocalContext
43 import androidx.compose.ui.res.painterResource
44 import androidx.compose.ui.res.stringResource
45 import androidx.compose.ui.text.font.FontWeight
46 import androidx.compose.ui.text.style.TextAlign
47 import androidx.compose.ui.text.style.TextOverflow
48 import androidx.compose.ui.tooling.preview.Preview
49 import androidx.compose.ui.unit.dp
50 import androidx.compose.ui.unit.sp
51 import androidx.navigation.NavController
52 import androidx.navigation.compose.NavHost
53 import androidx.navigation.compose.composable
54 import androidx.navigation.compose.rememberNavController
55 import
   com.example.modul3compose.ui.theme.Modul3ComposeTheme
56 import timber.log.Timber
57
58 @Composable
59 fun HotelApp(viewModel: HotelViewModel) {
60     val navController = rememberNavController()
61
62     val hotels by viewModel.listHotel.collectAsState()
63
64     NavHost(navController = navController,
startDestination = "hotel_list") {
65         composable("hotel_list") {
66             HotelList(
67                 hotels = hotels,

```

```

68         navController = navController,
69         onDetailClick = { hotel ->
70             viewModel.onHotelClicked(hotel)
71             navController.navigate("hotel_detail/${hotel.name}")
72         }
73     )
74 }
75
76     composable("hotel_detail/{hotelName}") {
77         backStackEntry ->
78             val hotelName =
79                 backStackEntry.arguments?.getString("hotelName")
80                 val hotel =
81                     viewModel.getHotelByName(hotelName)
82                     if (hotel != null) {
83                         HotelDetailScreen(
84                             hotel = hotel,
85                             onBackClick = {
86                                 navController.popBackStack()
87                             },
88                             viewModel = viewModel
89                         )
90                     }
91             }
92
93     composable("language_screen") {
94         LanguageScreen(onBackClick = {
95             navController.popBackStack()
96         })
97     }
98 }
99
100 @Composable
101 fun HotelList(
102     hotels: List<Hotel>,
103     navController: NavController,
104     onDetailClick: (Hotel) -> Unit
105 ) {
106     LazyColumn(modifier = Modifier.fillMaxSize()) {
107         item {
108             HomeHeader(navController = navController)
109
110             Text(
111                 text = stringResource(id =

```

```

112         MaterialTheme.typography.titleLarge,
113             fontWeight = FontWeight.ExtraBold
114         )
115         LazyRow(
116             contentPadding =
117             PaddingValues(horizontal = 8.dp),
118             horizontalArrangement =
119             Arrangement.spacedBy(8.dp)
120         ) {
121             items(hotels) { hotel ->
122                 HotelHighlightItem(
123                     hotel = hotel,
124                     onDetailClick = {
125                         onDetailClick(hotel) }
126                 )
127             }
128         }
129         Spacer(modifier = Modifier.height(16.dp))
130         Text(
131             text = stringResource(id =
132             R.string.all_hotels),
133             modifier = Modifier.padding(start =
134             16.dp, bottom = 8.dp),
135             style =
136             MaterialTheme.typography.titleMedium,
137             fontWeight = FontWeight.Bold
138         )
139     }
140     items(hotels) { hotel ->
141         HotelItemRow(
142             hotel = hotel,
143             onDetailClick = { onDetailClick(hotel) }
144         )
145     }
146 }
147
148 @Composable
149 fun HomeHeader(navController: NavController) {
150     Row(
151         modifier = Modifier
152         .fillMaxWidth()
153         .padding(16.dp),
154         horizontalArrangement = Arrangement.End,
155         verticalAlignment = Alignment.CenterVertically

```

```

154     ) {
155         IconButton(onClick = {
156             navController.navigate("language_screen") }) {
157             Icon(
158                 imageVector = Icons.Default.Language,
159                 contentDescription = "Change Language"
160             )
161         }
162     }
163
164     @Composable
165     fun HotelItemRow(hotel: Hotel, onDetailClick: () ->
166         Unit) {
167         val context = LocalContext.current
168         Card(
169             shape = RoundedCornerShape(16.dp),
170             modifier = Modifier
171                 .fillMaxWidth()
172                 .padding(8.dp),
173             elevation = CardDefaults.cardElevation(4.dp)
174         ) {
175             Row(modifier = Modifier.padding(12.dp),
176                 verticalAlignment = Alignment.CenterVertically) {
177                 Image(
178                     painter = painterResource(id =
179                         hotel.imageURL),
180                     contentDescription = null,
181                     modifier = Modifier
182                         .size(100.dp)
183                         .clip(RoundedCornerShape(12.dp)),
184                     contentScale = ContentScale.Crop
185                 )
186
187                 Spacer(modifier = Modifier.width(16.dp))
188
189                 Column(modifier = Modifier.weight(1f)) {
190                     Row(modifier = Modifier.fillMaxWidth(),
191                         horizontalArrangement = Arrangement.SpaceBetween) {
192                         Text(text = hotel.name, fontWeight =
193                             FontWeight.Bold, modifier = Modifier.weight(1f),
194                             overflow = TextOverflow.Ellipsis)
195                         Text(text = hotel.stars, modifier =
196                             Modifier.padding(start = 4.dp), fontSize = 12.sp)
197                     }
198
199                     Row(modifier = Modifier.fillMaxWidth(),
200                         horizontalArrangement = Arrangement.SpaceBetween) {
201                         Text(text = hotel.location, fontSize =

```

```

194         = 11.sp, color = Color.Gray)
195         Text(text = hotel.price, fontWeight
196         = FontWeight.SemiBold, color = Color.Blue, fontSize =
197         13.sp)
198     }
199     Spacer(modifier = Modifier.height(8.dp))
200     Row(
201         horizontalArrangement =
202         Arrangement.spacedBy(8.dp)
203     ) {
204         Button(
205             onClick = {
206                 Timber.tag("ACTION").d("Tombol
207                 Explicit Intent (Maps) ditekan untuk: ${hotel.name}")
208                 val uri =
209                 Uri.parse("geo:0,0?q=${hotel.name}+${hotel.location}")
210                 val mapIntent =
211                 Intent(Intent.ACTION_VIEW, uri)
212                 context.startActivity(mapIntent)
213             }
214         ) {
215             Text("Maps")
216         }
217         Button(
218             onClick = {
219                 Timber.tag("ACTION").d("Tombol
220                 Detail ditekan untuk: ${hotel.name}")
221                 onDetailClick()
222             }
223         ) {
224             Text("Detail")
225         }
226     }
227 }
228 @Composable
229 fun HotelHighlightItem(hotel: Hotel, onDetailClick: () -
230 > Unit) {
231     Card(
232         shape = RoundedCornerShape(20.dp),
233         modifier = Modifier
234             .width(300.dp)

```

```

234         .padding(8.dp)
235         .clip(RoundedCornerShape(20.dp))
236         .clickable { onDetailClick() },
237         elevation = CardDefaults.cardElevation(6.dp)
238     ) {
239         Column {
240             Image(
241                 painter = painterResource(id =
242 hotel.imageUrl),
243                 contentDescription = null,
244                 modifier = Modifier
245                     .fillMaxWidth()
246                     .height(150.dp)
247                     .clip(RoundedCornerShape(topStart =
248 20.dp, topEnd = 20.dp)),
249                 contentScale = ContentScale.Crop
250             )
251             Text(
252                 text = hotel.name,
253                 modifier = Modifier.padding(12.dp),
254                 fontWeight = FontWeight.Bold,
255                 style =
256                 MaterialTheme.typography.titleMedium
257             )
258         }
259     }
260 }
261
262 @Composable
263 fun HotelDetailScreen(hotel: Hotel, onBackClick: () ->
264 Unit, viewModel: HotelViewModel) {
265     Column(
266         modifier = Modifier
267             .fillMaxSize()
268             .verticalScroll(rememberScrollState())
269     ) {
270         Box {
271             Image(
272                 painter = painterResource(id =
273 hotel.imageUrl),
274                 contentDescription = null,
275                 modifier = Modifier
276                     .fillMaxWidth()
277                     .height(300.dp),
278                 contentScale = ContentScale.Crop
279             )
280
281             Button(
282                 onClick = onBackClick,

```

```

277         modifier = Modifier.padding(16.dp),
278         shape = RoundedCornerShape(50)
279     ) {
280         Text(text = stringResource(id =
R.string.back_button))
281     }
282 }
283
284     Column(modifier = Modifier.padding(16.dp)) {
285         Row(
286             modifier = Modifier.fillMaxWidth(),
287             horizontalArrangement =
Arrangement.SpaceBetween,
288             verticalAlignment =
Alignment.CenterVertically
289         ) {
290             Text(text = hotel.name, style =
MaterialTheme.typography.headlineMedium, fontWeight =
FontWeight.Bold, modifier = Modifier.weight(1f))
291             Spacer(modifier = Modifier.width(8.dp))
292             Text(text = hotel.stars, style =
MaterialTheme.typography.headlineSmall, textAlign =
TextAlign.End)
293         }
294
295         Text(text = hotel.location, color =
Color.Gray, fontSize = 16.sp)
296
297         Spacer(modifier = Modifier.height(16.dp))
298
299         Text(text = stringResource(id =
R.string.price_per_night), fontWeight =
FontWeight.SemiBold)
300         Text(text = hotel.price, color = Color.Blue,
fontSize = 24.sp, fontWeight = FontWeight.ExtraBold)
301
302         Spacer(modifier = Modifier.height(24.dp))
303
304         Text(
305             text = stringResource(id =
R.string.facilities_desc),
306             style =
MaterialTheme.typography.titleMedium,
307             fontWeight = FontWeight.Bold
308         )
309         Text(
310             text = stringResource(id =
R.string.hotel_description, hotel.name),
311             style =

```

```

312         MaterialTheme.typography.bodyMedium,
313             color = Color.DarkGray
314         )
315         Card(
316             modifier = Modifier.fillMaxWidth(),
317             colors =
318 CardDefaults.cardColors(containerColor =
319 Color(0xFFE8F5E9))
320         ) {
321             Text(
322                 text = stringResource(id =
323 R.string.promo_code, viewModel.promoCode),
324                 modifier = Modifier.padding(16.dp),
325                 fontWeight = FontWeight.Bold,
326                 color = Color(0xFF2E7D32)
327             )
328         }
329     }
330 }
331
332 @Preview(showBackground = true)
333 @Composable
334 fun HotelPreview() {
335     Modul3ComposeTheme() {
336         Column {
337             val sampleHotel = Hotel(
338                 "Kollektiv Hotel",
339                 "★★★",
340                 "Sukajadi, Bandung",
341                 "Rp423.649",
342                 R.drawable.kollektiv
343             )
344             HotelHighlightItem(hotel = sampleHotel,
345 onDetailClick = {})
346             Spacer(modifier = Modifier.height(20.dp))
347             HotelItemRow(hotel = sampleHotel,
348 onDetailClick = {})
349         }
350     }
351 }

```

HotelViewModel.kt

Tabel 5. Source Code HotelViewModel Compose Soal 1

```

1 package com.example.modul3compose
2
3 import androidx.lifecycle.ViewModel
4 import kotlinx.coroutines.flow.MutableStateFlow
5 import kotlinx.coroutines.flow.StateFlow
6 import kotlinx.coroutines.flow.asStateFlow
7 import timber.log.Timber
8
9 class HotelViewModel(val promoCode: String) :
  ViewModel() {
10     private val _listHotel = MutableStateFlow(
11         listOf(
12             Hotel("Kollektiv Hotel", "★★★★",
13 "Sukajadi, Bandung", "Rp423.649", R.drawable.kollektiv),
14             Hotel("eL Hotel Bandung", "★★★★★",
15 "Merdeka, Bandung", "Rp724.054", R.drawable.elhotel),
16             Hotel("Geary Hotel Bandung", "★★★★",
17 "Pasirkaliki, Bandung", "Rp387.061", R.drawable.geary),
18             Hotel("Pullman Bandung Grand Central",
19 "★★★★★", "Cibeunying, Bandung", "Rp1.652.504",
20 R.drawable.pullman),
21             Hotel("Grandia Hotel Bandung", "★★★★★",
22 "Cihampelas, Bandung", "Rp532.350", R.drawable.grandia)
23         )
24     )
25     val listHotel: StateFlow<List<Hotel>> =
26         _listHotel.asStateFlow()
27
28     private val _selectedHotel =
29         MutableStateFlow<Hotel?>(null)
30     val selectedHotel: StateFlow<Hotel?> =
31         _selectedHotel.asStateFlow()
32
33     fun getHotelByName(name: String?): Hotel? {
34         return _listHotel.value.find { it.name == name }
35     }
36
37     init {
38         Timber.d("DATA_LIST: Berhasil memuat
39 ${_listHotel.value.size} hotel ke dalam list.")
40     }
41
42     fun onHotelClicked(hotel: Hotel) {
43         _selectedHotel.value = hotel
44
45         Timber.i("NAVIGASI: User memilih hotel
46 ${hotel.name} untuk melihat detail.")
47     }
48 }

```

37	}
38	}

HotelViewModelFactory.kt

Tabel 6. Source Code HotelViewModelFactory Compose Soal 1

1	package com.example.modul3compose
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class HotelViewModelFactory(private val promo: String) :
7	ViewModelProvider.Factory {
8	override fun <T : ViewModel> create(modelClass:
9	Class<T>): T {
10	if
11	(modelClass.isAssignableFrom(HotelViewModel::class.java)
12	) {
13	@Suppress("UNCHECKED_CAST")
14	return HotelViewModel(promo) as T
15	}
16	throw IllegalArgumentException("Unknown
17	ViewModel class")
18	}
19	}

HotelFragment.kt

Tabel 7. Source Code HotelFragment Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.compose.ui.platform.ComposeView
8	import androidx.fragment.app.Fragment
9	import androidx.lifecycle.viewmodel.compose.viewModel
10	import
11	com.example.modul3compose.ui.theme.Modul3ComposeTheme
12	class HotelFragment : Fragment() {
13	
14	override fun onCreateView(
15	inflater: LayoutInflater,
16	container: ViewGroup?,

17	savedInstanceState: Bundle?
18	): View {
19	return ComposeView(requireContext()).apply {
20	setContent {
21	Modul3ComposeTheme {
22	val myFactory =
23	HotelViewModelFactory("PROMO-PELAJAR-2026")
24	val hotelViewModel: HotelViewModel =
25	viewModel(factory = myFactory)
26	HotelApp(viewModel = hotelViewModel)
27	}
28	}
29	}

LanguageScreen.kt

Tabel 8. Source Code LanguageScreen Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.content.res.Configuration
4	import androidx.compose.foundation.layout.Arrangement
5	import androidx.compose.foundation.layout.Column
6	import androidx.compose.foundation.layout.Spacer
7	import androidx.compose.foundation.layout.fillMaxSize
8	import androidx.compose.foundation.layout.fillMaxWidth
9	import androidx.compose.foundation.layout.height
10	import androidx.compose.foundation.layout.padding
11	import androidx.compose.foundation.rememberScrollState
12	import androidx.compose.foundation.verticalScroll
13	import androidx.compose.material3.Button
14	import androidx.compose.material3.MaterialTheme
15	import androidx.compose.material3.Text
16	import androidx.compose.material3.TextButton
17	import androidx.compose.runtime.Composable
18	import androidx.compose.ui.Alignment
19	import androidx.compose.ui.Modifier
20	import androidx.compose.ui.platform.LocalContext
21	import androidx.compose.ui.res.stringResource
22	import androidx.compose.ui.text.font.FontWeight
23	import androidx.compose.ui.unit.dp
24	import androidx.lifecycle.viewmodel.compose.viewModel
25	import java.util.Locale
26	
27	@Composable
28	fun LanguageScreen(
29	onBackClick: () -> Unit,

```

30     languageViewModel: LanguageViewModel = viewModel()
31 ) {
32     val context = LocalContext.current
33
34     Column(
35         modifier = Modifier
36             .fillMaxSize()
37             .padding(16.dp)
38             .verticalScroll(rememberScrollState()),
39         verticalArrangement = Arrangement.Center,
40         horizontalAlignment =
41 Alignment.CenterHorizontally
42     ) {
43         Text(
44             text = stringResource(id =
45 R.string.language_selector),
46             style =
47 MaterialTheme.typography.titleLarge,
48             fontWeight = FontWeight.Bold
49         )
50
51         Spacer(modifier = Modifier.height(32.dp))
52
53         languageViewModel.LanguageList.forEach {
54 language ->
55             Button(
56                 onClick = {
57                     val locale = Locale(language.code)
58                     Locale.setDefault(locale)
59                     val config = Configuration()
60                     config.setLocale(locale)
61                     context.resources.updateConfiguration(config,
62 context.resources.displayMetrics)
63
64                     onBackClick()
65                 },
66                 modifier = Modifier
67                     .fillMaxWidth()
68                     .padding(horizontal = 32.dp)
69             ) {
70                 Text(text = language.name)
71             }
72
73             Spacer(modifier = Modifier.height(12.dp))
74         }
75
76         Spacer(modifier = Modifier.height(20.dp))
77
78         TextButton(onClick = onBackClick) {
79             Text(text = stringResource(id =

```

73	R.string.back_button))
74	}
75	}
76	}

LanguageViewModel.kt

Tabel 9. Source Code LanguageViewModel Compose Soal 1

1	package com.example.modul3compose
2	
3	import androidx.lifecycle.ViewModel
4	
5	class LanguageVliewModel : ViewModel() {
6	val LanguageList = listOf(
7	LanguageItem("Bahasa Indonesia", "id"),
8	LanguageItem("English", "en"),
9	)
10	}
11	
12	data class LanguageItem(
13	val name: String,
14	val code: String
15	)

XML

MainActivity.kt

Tabel 10. Source Code Mainactivity XML Soal 1

```
1 package com.example.modul3
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import com.example.modul3.databinding.ActivityMainBinding
6
7 class MainActivityXml : AppCompatActivity() {
8
9     private lateinit var binding: ActivityMainBinding
10
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13
14         binding =
15 ActivityMainBinding.inflate(layoutInflater)
16         setContentView(binding.root)
17
18         if (savedInstanceState == null) {
19             supportFragmentManager.beginTransaction()
20                 .replace(R.id.fragment_container,
21 HomeFragment())
22                 .commit()
23         }
24     }
25 }
```

MainApplication.kt

Tabel 11. Source Code MainApplication XML Soal 1

```
1 package com.example.modul3
2
3 import android.app.Application
4 import timber.log.Timber
5
6 class MyApplication : Application() {
7     override fun onCreate() {
8         super.onCreate()
9         if
10 (android.content.pm.ApplicationInfo.FLAG_DEBUGGABLE !=
11 0) {
12             Timber.plant(Timber.DebugTree())
13         }
14     }
15 }
```

13	} }
----	--------

HotelViewModel.kt

Tabel 12. Source Code HotelViewModel XML Soal 1

1	package com.example.modul3
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	import kotlinx.coroutines.flow.asStateFlow
8	import kotlinx.coroutines.launch
9	
10	class HotelViewModel(private val promoCode: String) : ViewModel() {
11	private val _hotels = MutableStateFlow<List<Hotel>>(emptyList())
12	
13	private val _navigateToDetail = MutableStateFlow<Hotel?>(null)
14	val navigateToDetail = _navigateToDetail.asStateFlow()
15	val hotels: StateFlow<List<Hotel>> = _hotels.asStateFlow()
16	
17	fun onHotelClicked(hotel: Hotel) {
18	timber.log.Timber.d("LOG_EVENT: Tombol Detail ditekan. Data terpilih: Name=\${hotel.name}, Price=\${hotel.price}")
19	
20	_navigateToDetail.value = hotel
21	}
22	
23	fun onDetailNavigated() {
24	_navigateToDetail.value = null
25	}
26	init {
27	loadHotels()
28	}
29	
30	private fun loadHotels() {
31	val list = listOf(32
	Hotel("Kollektiv Hotel", "★★★★", "Sukajadi, Bandung", "Rp423.649", listOf(R.drawable.kollektiv)),
33	Hotel("eL Hotel Bandung", "★★★★★",

	"Merdeka, Bandung", "Rp724.054", listOf(R.drawable.elhotel)),
34	Hotel("Geary Hotel Bandung", "★★★★", "Pasirkaliki, Bandung", "Rp387.061", listOf(R.drawable.geary)),
35	Hotel("Pullman Bandung Grand Central", "★★★★★", "Cibeunying, Bandung", "Rp1.652.504", listOf(R.drawable.pullman)),
36	Hotel("Grandia Hotel Bandung", "★★★★★", "Cihampelas, Bandung", "Rp532.350", listOf(R.drawable.grandia))
37	)
38	_hotels.value = list
39	timber.log.Timber.d("LOG_EVENT: Berhasil memuat
40	\${list.size} data hotel ke dalam list.")
41	}
42	}

HotelViewModelFactory.kt

Tabel 13. Source Code HotelViewModelFactory XML Soal 1

1	package com.example.modul3
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class HotelViewModelFactory(private val promoCode: String) : ViewModelProvider.Factory {
7	override fun <T : ViewModel> create(modelClass: Class<T>): T {
8	if
	(modelClass.isAssignableFrom(HotelViewModel::class.java)
	) {
9	return HotelViewModel(promoCode) as T
10	}
11	throw IllegalArgumentException("Unknown
12	ViewModel class")
	}
13	}

HotelAdapter.kt

Tabel 14. Source Code HotelAdapter.kt XML Soal 1

1	package com.example.modul3
2	

```

3 import android.content.Intent
4 import android.net.Uri
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import android.widget.Button
9 import android.widget.ImageView
10 import android.widget.RatingBar
11 import android.widget.TextView
12 import androidx.recyclerview.widget.RecyclerView
13
14 class HotelAdapter(
15     private val listHotel: List<Hotel>,
16     private val onItemClick: (Hotel) -> Unit
17 ) : RecyclerView.Adapter<HotelAdapter.ListViewHolder>() {
18
19     class ListViewHolder(itemView: View) :
20 RecyclerView.ViewHolder(itemView) {
21         val tvName: TextView =
22 itemView.findViewById(R.id.tv_hotel_name)
23         val tvLocation: TextView =
24 itemView.findViewById(R.id.tv_hotel_location)
25         val tvPrice: TextView =
26 itemView.findViewById(R.id.tv_hotel_price)
27         val ratingHotel: RatingBar =
28 itemView.findViewById(R.id.rating_hotel)
29         val btnMaps: Button =
30 itemView.findViewById(R.id.btn_maps)
31         val btnDetail: Button =
32 itemView.findViewById(R.id.btn_detail)
33         val viewPagerImage:
34 androidx.viewpager2.widget.ViewPager2 =
35 itemView.findViewById(R.id.viewPagerImage)
36     }
37
38     override fun onCreateViewHolder(parent: ViewGroup,
39 viewType: Int): ListViewHolder {
40         val view: View =
41 LayoutInflater.from(parent.context).inflate(R.layout.item
42 _hotel, parent, false)
43         return ListViewHolder(view)
44     }
45
46     override fun onBindViewHolder(holder: ListViewHolder,
47 position: Int) {
48         val hotel = listHotel[position]
49         val context = holder.itemView.context
50         val carouselAdapter = ImageAdapter(listOf(hotel),
51 false, onItemClick)

```

```

38
39         holder.viewPagerImage.adapter = carouselAdapter
40         holder.tvName.text = hotel.name
41         holder.tvLocation.text = hotel.location
42         holder.tvPrice.text = hotel.price
43         holder.ratingHotel.rating =
hotel.stars.length.toFloat()
44
45         holder.btnMaps.setOnClickListener {
46             timber.log.Timber.d("LOG_EVENT: Tombol Maps
(Explicit Intent) diklik untuk hotel: ${hotel.name}")
47
48             val query = "geo:0,0?q=${hotel.name}
${hotel.location}"
49             val intent = Intent(Intent.ACTION_VIEW,
Uri.parse(query))
50             context.startActivity(intent)
51         }
52
53         holder.btnDetail.setOnClickListener {
54             timber.log.Timber.d("LOG_EVENT: Tombol Detail
diklik untuk hotel: ${hotel.name}")
55             onItemClick(hotel)
56         }
57     }
58
59     override fun getItemCount(): Int = listHotel.size
60 }
61
62 class ImageAdapter(
63     private val listHotel: List<Hotel>,
64     private val showText: Boolean = false,
65     private val onItemClick: (Hotel) -> Unit
66 ) : RecyclerView.Adapter<ImageAdapter.ImageViewHolder>()
{
67
68     class ImageViewHolder(view: View) :
RecyclerView.ViewHolder(view) {
69         val img: ImageView =
view.findViewById(R.id.imageView)
70         val tvName: TextView =
view.findViewById(R.id.tvHotelName)
71     }
72
73     override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): ImageViewHolder {
74         val view =
LayoutInflater.from(parent.context).inflate(R.layout.view
pager_item, parent, false)

```

75	return ViewHolder(view)
76	}
77	
78	override fun onBindViewHolder(holder: ViewHolder, position: Int) {
79	val hotel = listHotel[position]
80	
81	holder.img.setImageResource(hotel.images[0])
82	
83	if (showText) {
84	holder.tvName.visibility = View.VISIBLE
85	holder.tvName.text = hotel.name
86	} else {
87	holder.tvName.visibility = View.GONE
88	}
89	
90	holder.itemView.setOnClickListener {
91	timber.log.Timber.d("LOG_EVENT: Item
92	`\${hotel.name}` diklik. Navigasi ke Detail dijalankan.")
93	onItemClick(hotel)
94	}
95	}
96	
97	override fun getItemCount(): Int = listHotel.size
98	}

DetailActivity.kt

Tabel 15. Source Code DetailActivity XML Soal 1

1	package com.example.modul3
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.lifecycle.ViewModelProvider
6	import
7	com.example.modul3.databinding.ActivityDetailBinding
8	class DetailActivity : AppCompatActivity() {
9	
10	private lateinit var binding: ActivityDetailBinding
11	private lateinit var viewModel: DetailViewModel
12	
13	override fun onCreate(savedInstanceState: Bundle?)
14	{
15	super.onCreate(savedInstanceState)
16	binding =

17	ActivityDetailBinding.inflate(layoutInflater)
18	setContentView(binding.root)
19	val name =
20	intent.getStringExtra("EXTRA_HOTEL_NAME")
21	val image =
22	intent.getIntExtra("EXTRA_HOTEL_IMAGE", 0)
23	val rating =
24	intent.getStringExtra("EXTRA_HOTEL_RATING")
25	val location =
26	intent.getStringExtra("EXTRA_HOTEL_LOCATION")
27	val price =
28	intent.getStringExtra("EXTRA_HOTEL_PRICE")
29	val desc =
30	intent.getStringExtra("EXTRA_HOTEL_DESC")
31	binding.tvNameDetail.text = name
32	binding.imgDetail.setImageResource(image)
33	binding.tvRatingDetail.text = rating
34	binding.tvLocationDetail.text = location
35	binding.tvPriceDetail.text = price
36	binding.tvDescDetail.text = desc
37	val factory = ViewModelFactory("PROMO_PELAJAR")
38	viewModel = ViewModelProvider(this,
39	factory)[DetailViewModel::class.java]
40	viewModel.promo.observe(this) { promoText ->
41	binding.tvPromoLabel.text =
42	"\${getString(R.string.promo_code)} \$promoText"
43	}
44	binding.btnBackDetail.setOnClickListener {
45	onBackPressedDispatcher.onBackPressed()
46	}
47	}
48	override fun onSupportNavigateUp(): Boolean {
	onBackPressedDispatcher.onBackPressed()
	return true
	}

activity_main.xml

Tabel 16. Source Code activity_main XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout

3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent">
7	
8	<FrameLayout
9	android:id="@+id/fragment_container"
10	android:layout_width="match_parent"
11	android:layout_height="match_parent" />
12	
13	</androidx.constraintlayout.widget.ConstraintLayout>

activity_detail.xml

Tabel 17. Source Code activity_detail. XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:background="#FFFFFF">
7	
8	<LinearLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content"
11	android:orientation="vertical">
12	
13	<FrameLayout
14	android:layout_width="match_parent"
15	android:layout_height="250dp">
16	
17	<ImageView
18	android:id="@+id/img_detail"
19	android:layout_width="match_parent"
20	android:layout_height="match_parent"
21	android:scaleType="centerCrop"
22	android:src="@drawable/kollektiv" />
23	
24	<com.google.android.material.button.MaterialButton
25	android:id="@+id/btn_back_detail"
26	android:layout_width="wrap_content"
27	android:layout_height="wrap_content"
28	android:layout_margin="16dp"
29	android:padding="12dp"
30	android:text="@string/back_button"

31	app:iconPadding="0dp"
32	android:textColor="#21005D"
33	app:iconTint="#21005D"
34	app:backgroundTint="@color/purple_200"
35	app:cornerRadius="24dp"
36	app:elevation="4dp" />
37	</FrameLayout>
38	
39	<LinearLayout
40	android:layout_width="match_parent"
41	android:layout_height="wrap_content"
42	android:orientation="vertical"
43	android:padding="20dp">
44	
45	<RelativeLayout
46	android:layout_width="match_parent"
47	android:layout_height="wrap_content">
48	
49	<LinearLayout
50	android:layout_width="wrap_content"
51	android:layout_height="wrap_content"
52	android:orientation="vertical"
53	android:layout_alignParentStart="true"
54	android:layout_toStartOf="@id/tv_rating_detail"
55	android:layout_marginEnd="16dp">
56	
57	<TextView
58	android:id="@+id/tv_name_detail"
59	android:layout_width="wrap_content"
60	android:layout_height="wrap_content"
61	android:text="Hotel Name"
62	android:textSize="22sp"
63	android:textStyle="bold"
64	android:textColor="@color/black"
65	/>
66	
67	<TextView
68	android:id="@+id/tv_location_detail"
69	android:layout_width="wrap_content"
70	android:layout_height="wrap_content"
71	android:text="Location Name"
72	android:textSize="14sp"
73	android:textColor="#888888" />
74	</LinearLayout>
75	<TextView
76	android:id="@+id/tv_rating_detail"
77	android:layout_width="wrap_content"
78	android:layout_height="wrap_content"

```

79         android:layout_alignParentEnd="true"
80         android:layout_centerVertical="true"
81         android:text="★★★"
82         android:textSize="16sp" />
83     </RelativeLayout>
84
85     <LinearLayout
86         android:layout_width="match_parent"
87         android:layout_height="wrap_content"
88         android:orientation="vertical"
89         android:layout_marginTop="24dp">
90
91         <TextView
92             android:layout_width="wrap_content"
93             android:layout_height="wrap_content"
94             android:text="@string/price_per_night"
95             android:textSize="14sp"
96             android:textColor="#666666" />
97
98         <TextView
99             android:id="@+id/tv_price_detail"
100             android:layout_width="wrap_content"
101             android:layout_height="wrap_content"
102             android:text="Rp 0"
103             android:textSize="20sp"
104             android:textStyle="bold"
105             android:textColor="#21005D" />
106     </LinearLayout>
107
108     <View
109         android:layout_width="match_parent"
110         android:layout_height="1dp"
111         android:background="#EEEEEE"
112         android:layout_marginTop="20dp"
113         android:layout_marginBottom="20dp" />
114
115     <TextView
116         android:layout_width="match_parent"
117         android:layout_height="wrap_content"
118         android:text="@string/facilities_desc"
119         android:textSize="16sp"
120         android:textStyle="bold"
121         android:textColor="@color/black" />
122
123     <TextView
124         android:id="@+id/tv_desc_detail"
125         android:layout_width="match_parent"
126         android:layout_height="wrap_content"
127         android:layout_marginTop="8dp"

```

128	android:textSize="15sp"
129	android:lineSpacingExtra="6dp"
130	android:textColor="#444444"
131	android:text="@string/hotel_description"
	/>
132	
133	<TextView
134	android:id="@+id/tv_promo_label"
135	android:layout_width="match_parent"
136	android:layout_height="wrap_content"
137	android:layout_marginTop="8dp"
138	android:background="#E8F5E9"
139	android:padding="16dp"
140	android:textColor="#2E7D32"
141	android:textStyle="bold"
142	android:text="@string/promo_code" />
143	</LinearLayout>
144	</LinearLayout>
145	</ScrollView>

item_hotel.xml

Tabel 18. Source Code item_hotel XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="12dp"
8	app:cardCornerRadius="20dp"
9	app:cardElevation="8dp"
10	app:cardMaxElevation="10dp"
11	app:cardPreventCornerOverlap="false"
12	app:cardBackgroundColor="#F5F5F5"
13	app:strokeColor="#D3D3D3"
14	app:strokeWidth="1dp">
15	
16	<LinearLayout
17	android:layout_width="match_parent"
18	android:layout_height="wrap_content"
19	android:orientation="vertical"
20	android:padding="12dp">
21	
22	<LinearLayout
23	android:layout_width="match_parent"
24	android:layout_height="wrap_content"

25	android:orientation="horizontal">
26	
27	<androidx.viewpager2.widget.ViewPager2
28	android:id="@+id/viewPagerImage"
29	android:layout_width="100dp"
30	android:layout_height="100dp"
31	android:layout_gravity="center_vertical"
	/>
32	
33	<LinearLayout
34	android:layout_width="0dp"
35	android:layout_height="wrap_content"
36	android:layout_weight="1"
37	android:layout_marginStart="16dp"
38	android:orientation="vertical">
39	
40	<LinearLayout
41	android:layout_width="match_parent"
42	android:layout_height="wrap_content"
43	android:orientation="horizontal">
44	
45	<TextView
46	android:id="@+id/tv_hotel_name"
47	android:layout_width="0dp"
48	android:layout_height="wrap_content"
49	android:layout_weight="1"
50	android:text="Hotel Name"
51	android:textSize="16sp"
52	android:textStyle="bold"
53	android:textColor="@color/black"/>
54	
55	<RatingBar
56	android:id="@+id/rating_hotel"
57	style="?android:attr/ratingBarStyleSmall"
58	android:layout_width="wrap_content"
59	android:layout_height="wrap_content"
60	android:numStars="5"
61	android:stepSize="0.5"
62	android:isIndicator="true"
63	android:progressTint="#FFB300"
64	android:layout_gravity="end" />
65	</LinearLayout>
66	
67	<TextView
68	android:id="@+id/tv_hotel_location"
69	android:layout_width="match_parent"
70	android:layout_height="wrap_content"
71	android:text="Location"
72	android:textColor="@android:color/darker_gray"

73	android:textSize="12sp" />
74	
75	<TextView
76	android:id="@+id/tv_hotel_price"
77	android:layout_width="match_parent"
78	android:layout_height="wrap_content"
79	android:layout_marginTop="4dp"
80	android:text="Price"
81	android:textColor="#0000FF"
82	android:textStyle="bold"
83	android:gravity="end"/>
84	</LinearLayout>
85	</LinearLayout>
86	
87	<LinearLayout
88	android:layout_width="match_parent"
89	android:layout_height="wrap_content"
90	android:layout_marginTop="12dp"
91	android:orientation="horizontal"
92	android:gravity="center">
93	
94	<Button
95	android:id="@+id/btn_maps"
96	android:layout_width="wrap_content"
97	android:layout_height="wrap_content"
98	android:text="Maps"
99	android:backgroundTint="@color/purple_200"
100	android:textColor="@color/white"/>
101	
102	<Button
103	android:id="@+id/btn_detail"
104	android:layout_width="wrap_content"
105	android:layout_height="wrap_content"
106	android:layout_marginStart="8dp"
107	android:text="Detail"
108	android:backgroundTint="@color/purple_200"
109	android:textColor="@color/white"/>
110	</LinearLayout>
111	
112	</LinearLayout>
113	</com.google.android.material.card.MaterialCardView>

fragment_home.xml

Tabel 19. Source Code fragment_home XML Soal 1

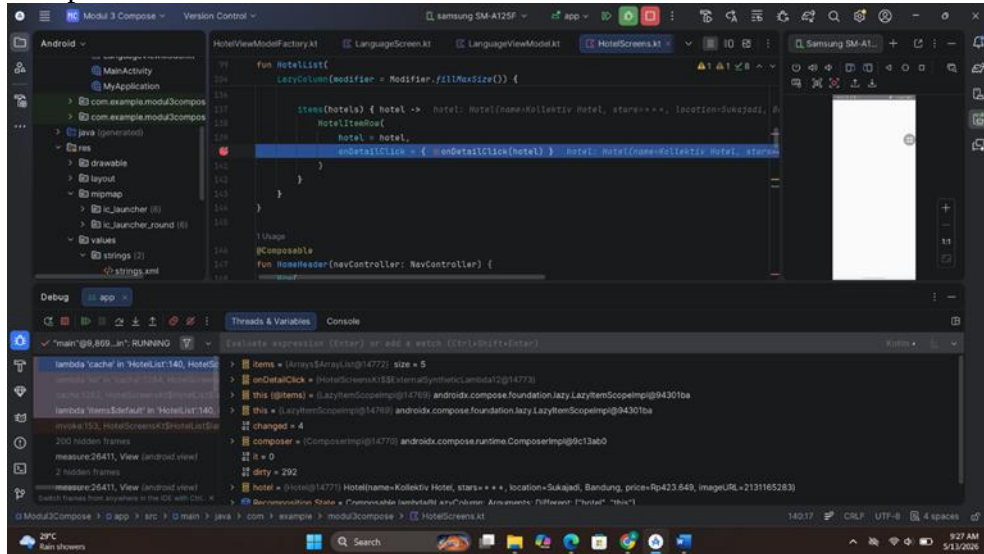
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.core.widget.NestedScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"

3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:background="#F8F9FA">
7	
8	<LinearLayout
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content"
11	android:orientation="vertical">
12	
13	<RelativeLayout
14	android:layout_width="match_parent"
15	android:layout_height="wrap_content"
16	android:padding="16dp">
17	
18	<TextView
19	android:layout_width="wrap_content"
20	android:layout_height="wrap_content"
21	android:layout_alignParentStart="true"
22	android:layout_centerVertical="true"
23	android:text="@string/featured_hotels"
24	android:textSize="20sp"
25	android:textStyle="bold"
26	android:textColor="@color/black"/>
27	
28	<TextView
29	android:id="@+id/btn_language"
30	android:layout_width="48dp"
31	android:layout_height="48dp"
32	android:layout_alignParentEnd="true"
33	android:layout_centerVertical="true"
34	android:text="🌐"
35	android:textSize="22sp"
36	android:gravity="center"
37	android:clickable="true"
38	android:focusable="true"
39	android:contentDescription="@string/language_selector"
40	/>
41	</RelativeLayout>
42	<FrameLayout
43	android:layout_width="match_parent"
44	android:layout_height="220dp"
45	android:layout_marginStart="16dp"
46	android:layout_marginEnd="16dp"
47	android:layout_marginTop="8dp">
48	
49	<androidx.viewpager2.widget.ViewPager2
50	android:id="@+id/viewPagerHeader"

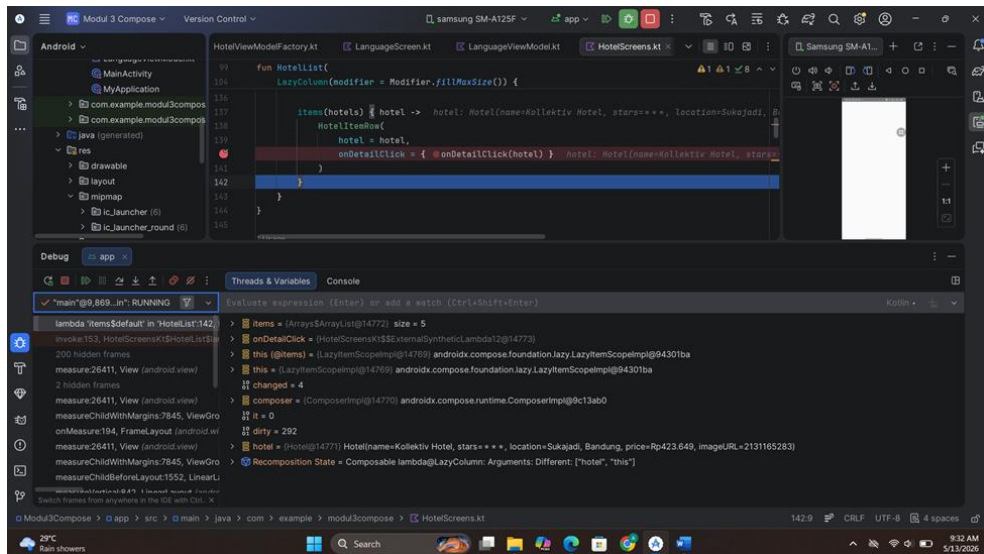
51	android:layout_width="match_parent"
52	android:layout_height="match_parent" />
53	
54	<LinearLayout
55	android:id="@+id/dotsContainer"
56	android:layout_width="wrap_content"
57	android:layout_height="wrap_content"
58	android:layout_gravity="bottom center"
59	android:layout_marginEnd="16dp"
60	android:layout_marginBottom="16dp"
61	android:background="#40000000"
62	android:clipToPadding="false"
63	android:orientation="horizontal"
64	android:padding="4dp" />
65	</FrameLayout>
66	
67	<TextView
68	android:layout_width="wrap_content"
69	android:layout_height="wrap_content"
70	android:text="@string/all_hotels"
71	android:textSize="18sp"
72	android:textStyle="bold"
73	android:textColor="@color/black"
74	android:padding="16dp"
75	android:layout_marginTop="16dp"/>
76	
77	<androidx.recyclerview.widget.RecyclerView
78	android:id="@+id/rv_hotels"
79	android:layout_width="match_parent"
80	android:layout_height="wrap_content"
81	android:nestedScrollingEnabled="false"
82	android:padding="8dp"/>
83	</LinearLayout>
84	</androidx.core.widget.NestedScrollView>

B. Output Program

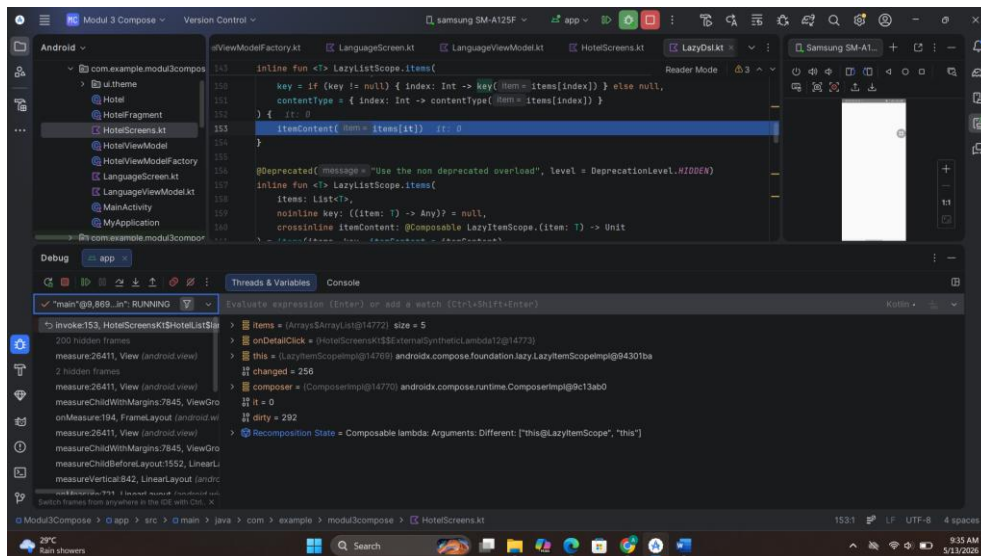
A. Compose



Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose (Step Into)

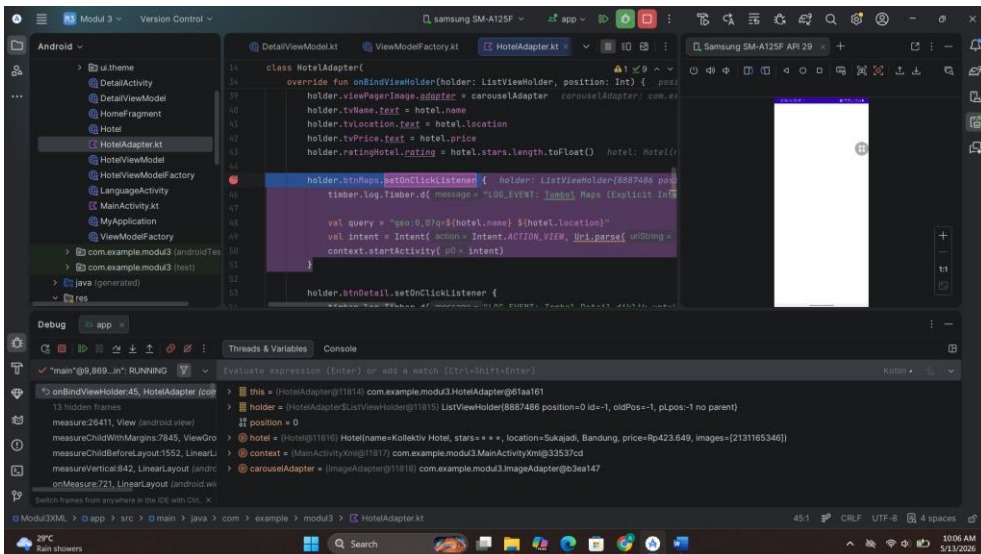


Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose (Step Over)

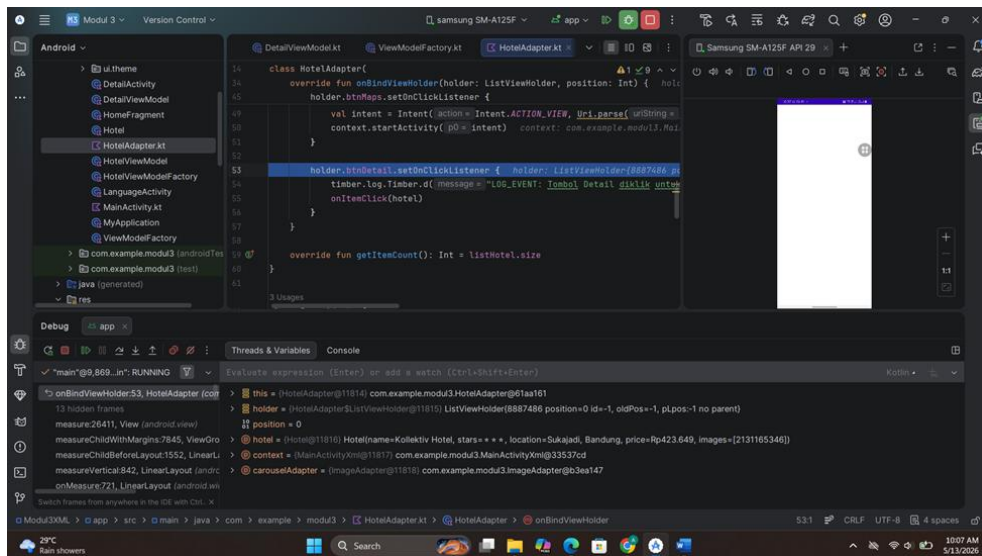


Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose (Step Out)

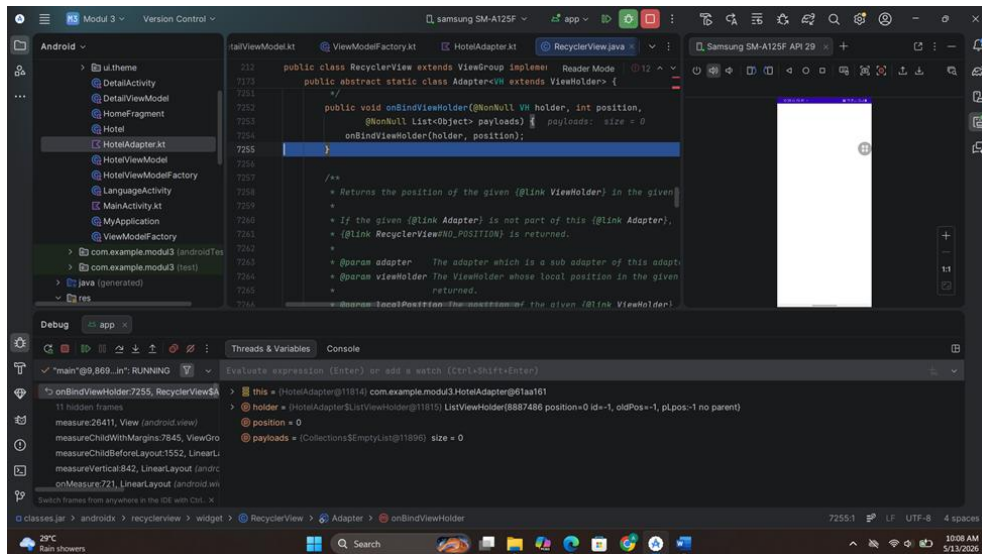
B. XML



Gambar 4. Screenshot Hasil Jawaban Soal 1 XML (Step Into)



Gambar 5. Screenshot Hasil Jawaban Soal 1 XML (Step Over)



Gambar 6. Screenshot Hasil Jawaban Soal 1 XML (Step Out)

C. Pembahasan

COMPOSE

MainActivity.kt

File ini merupakan komponen utama yang menjalankan fungsi "onCreate(savedInstanceState: Bundle?)" dengan parameter "savedInstanceState" untuk menangani siklus hidup aktivitas. Fungsi yang paling krusial di sini adalah "setContent", di mana terdapat pemanggilan "rememberNavController()" tanpa parameter untuk menginisialisasi kontroler navigasi. Di dalam blok ini, terdapat "NavHost" yang menggunakan parameter "navController" dan "startDestination" untuk menentukan rute awal aplikasi, seperti daftar hotel atau pengaturan bahasa, sehingga perpindahan antar layar dapat terorganisir.

MyApplication.kt

File ini adalah kelas yang mewarisi "Application" dan menjalankan fungsi "onCreate()" secara otomatis saat aplikasi pertama kali dibuat oleh sistem. Meskipun tidak menggunakan parameter dalam deklarasinya, fungsi ini sangat krusial karena berfungsi sebagai tempat inisialisasi global. Kode di sini memastikan bahwa semua konfigurasi dasar aplikasi sudah siap sebelum aktivitas apa pun muncul di layar.

Hotel.kt

File ini berisi "data class Hotel" yang mendefinisikan struktur objek dengan parameter konstruktor berupa "name: String", "stars: String", "location: String", "price: String", dan "imageUrl: Int". Parameter "imageUrl" bertipe "Int" sangat krusial karena digunakan untuk menyimpan referensi ID gambar dari folder resource drawable. Data class ini memudahkan pengelolaan informasi hotel saat data tersebut dioper antar fungsi atau ditampilkan ke dalam list UI.

HotelScreens.kt

File ini berisi fungsi-fungsi UI berbasis Compose, salah satunya "HotelList" yang

menerima parameter daftar hotel dan fungsi lambda "onDetailClick". Fungsi yang sering muncul di sini adalah "LazyColumn" dengan "items()" untuk menampilkan daftar secara efisien. Untuk fitur peta, digunakan implicit intent melalui fungsi "Intent(Intent.ACTION_VIEW, Uri.parse(uri))", di mana "Uri.parse" mengubah string alamat menjadi URI agar bisa dieksekusi oleh "context.startActivity(intent)" untuk membuka aplikasi peta eksternal.

HotelViewModel.kt

File ini berfungsi mengelola data menggunakan fungsi seperti "selectHotel(hotel: Hotel)" yang menerima parameter objek hotel yang diklik oleh pengguna. Bagian krusialnya adalah penggunaan "mutableStateOf" atau "StateFlow" untuk membungkus data hotel. Fungsi ini sangat krusial karena saat nilai statusnya berubah, Jetpack Compose akan otomatis memperbarui tampilan detail tanpa perlu memanggil fungsinya secara manual.

HotelViewModelFactory.kt

File ini menyediakan fungsi "create(modelClass: Class): T" yang menerima parameter tipe kelas ViewModel yang ingin dibuat. Fungsi ini krusial untuk melakukan instansiasi ViewModel yang memiliki parameter pada konstruktornya. Di dalamnya terdapat pengecekan menggunakan "modelClass.isAssignableFrom" untuk memastikan bahwa kelas yang dibuat benar-benar turunan dari ViewModel yang diinginkan sebelum dikembalikan ke UI.

HotelFragment.kt

Fungsi utama yang digunakan di sini adalah "onCreateView" yang menerima parameter "inflater", "container", dan "savedInstanceState" untuk mengatur layout fragment. Hal krusial dalam file ini adalah penggunaan "ComposeView" yang memanggil fungsi "setContent". Ini memungkinkan penempatan tampilan berbasis Jetpack Compose ke dalam sistem Fragment yang merupakan bagian dari arsitektur Android klasik.

LanguageScreen.kt

File ini berisi fungsi "@Composable" yang menampilkan UI untuk pengaturan bahasa dengan parameter yang diambil dari ViewModel. Fungsi yang sering muncul melibatkan komponen seperti "RadioButton" atau "Card" yang memiliki parameter status pilihan. Saat terjadi interaksi pengguna, fungsi ini akan memanggil metode di ViewModel untuk memproses pergantian bahasa yang diinginkan.

LanguageViewModel.kt

File ini memiliki fungsi utama "setLanguage(languageCode: String)" yang menerima parameter string berupa kode bahasa. Fungsi krusial yang digunakan adalah "AppCompatActivity.setApplicationLocales()" dengan parameter dari "LocaleListCompat.forLanguageTags(languageCode)". Fungsi-fungsi ini bekerja untuk mengganti konfigurasi bahasa aplikasi secara sistemik tanpa perlu memulai ulang aktivitas secara manual.

XML

MainActivityXml.kt

Fungsi "onCreate" dengan parameter "savedInstanceState" merupakan titik awal aktivitas untuk mengatur tampilan melalui "setContentView". Di dalamnya terdapat logika "supportFragmentManager.beginTransaction" yang memanggil fungsi "replace" dengan parameter "R.id.fragment_container" dan "HomeFragment()". Hal ini krusial untuk memastikan bahwa saat aplikasi dibuka, "HomeFragment" langsung dimuat ke dalam wadah yang sudah disediakan di layout XML.

MyApplication.kt

File ini mewarisi kelas "Application" dan menjalankan fungsi "onCreate" sebagai prosedur standar saat proses aplikasi dimulai. Di dalamnya terdapat pemanggilan fungsi "Timber.plant" yang menerima parameter "Timber.DebugTree()". Fungsi ini berfungsi untuk menginisialisasi pustaka logging "Timber" sehingga pengembang dapat melihat log aktivitas aplikasi di konsol selama tahap pengembangan.

HotelViewModel.kt

File ini mengelola data dan logika bisnis menggunakan "StateFlow". Fungsi "loadHotels" digunakan untuk mengisi daftar data hotel ke dalam variabel "_hotels". Selain itu, terdapat fungsi "onHotelClicked" yang menerima parameter objek "hotel". Fungsi ini bertugas memperbarui variabel "navigateToDetail" dengan data hotel yang diklik, sehingga fragment yang mengamati data ini dapat mengetahui kapan harus berpindah ke halaman detail.

HotelViewModelFactory.kt

File ini menyediakan fungsi "create(modelClass: Class): T" yang menerima parameter tipe kelas ViewModel yang ingin dibuat. Fungsi ini sangat penting karena "HotelViewModel" memerlukan parameter konstruktor berupa "promoCode". Di dalamnya terdapat pengecekan menggunakan fungsi "modelClass.isAssignableFrom" untuk memvalidasi bahwa kelas yang diminta

adalah "HotelViewModel" sebelum akhirnya mengembalikan instance ViewModel tersebut dengan parameter yang sesuai.

HotelAdapter.kt

Fungsi "onCreateViewHolder" menggunakan parameter "parent" untuk memompa layout "item_hotel.xml" menggunakan "ItemHotelBinding". Fungsi "onBindViewHolder" menerima parameter "holder" dan "position" untuk menghubungkan data dari list ke elemen UI spesifik. Di dalam fungsi ini, terdapat pemanggilan "setOnClickListener" pada root view yang memicu fungsi "onHotelClicked" dari ViewModel, sehingga setiap item dalam "RecyclerView" dapat merespons sentuhan pengguna.

DetailActivity.kt

Fungsi "onCreate" menggunakan parameter "savedInstanceState" untuk memulai aktivitas. Hal paling krusial di sini adalah penggunaan fungsi "intent.getStringExtra" dan "intent.getIntExtra" yang menerima parameter kunci seperti "EXTRA_HOTEL_NAME" atau "EXTRA_HOTEL_IMAGE". Data yang ditangkap dari intent tersebut kemudian langsung diterapkan ke elemen "TextView" dan "ImageView" melalui objek binding agar informasi hotel yang dipilih muncul dengan lengkap di layar detail.

activity_main.xml

File ini menggunakan komponen "FrameLayout" yang memiliki parameter "android:id" dengan nilai "fragment_container". Fungsi utamanya adalah menyediakan area kosong di dalam layout aktivitas yang akan diisi secara dinamis oleh "HomeFragment" melalui "FragmentManager" di kode Kotlin.

activity_detail.xml

Layout ini mendefinisikan struktur halaman detail menggunakan "ImageView" untuk gambar hotel besar dan beberapa "TextView" untuk nama serta harga. Setiap elemen

memiliki parameter "android:id" agar "DetailActivity" dapat mengakses dan mengubah isinya sesuai dengan data hotel yang dikirim melalui intent.

item_hotel.xml

File ini menyediakan desain untuk setiap baris di dalam list menggunakan "CardView". Terdapat parameter layout seperti "android:layout_width" dan "android:layout_height" yang diatur sedemikian rupa agar tampilan kartu hotel terlihat konsisten. Di dalamnya terdapat "ImageView" dan "TextView" yang akan diisi datanya secara berulang oleh "HotelAdapter".

fragment_home.xml

File ini menggunakan "NestedScrollView" agar konten di dalamnya bisa digulir secara vertikal. Komponen utamanya adalah "RecyclerView" dengan parameter "android:id" yaitu "rv_hotels". Fungsi dari elemen ini adalah sebagai wadah tempat "HotelAdapter" akan merender banyak item hotel secara efisien dalam bentuk daftar.

Tool Debugger:

Debugger adalah sebuah tools yang digunakan untuk membantu programmer menemukan dan memperbaiki bug atau kesalahan pada program. Dengan debugger, programmer dapat menjalankan program secara bertahap untuk melihat alur eksekusi kode, memeriksa nilai variabel, serta mengetahui bagian program yang tidak berjalan sebagaimana mestinya.

Cara menggunakan debugger biasanya dilakukan dengan menambahkan breakpoint pada baris kode tertentu yang ingin diperiksa. Setelah itu, program dijalankan dalam mode debug sehingga eksekusi program akan berhenti sementara pada breakpoint tersebut. Dari sini, programmer dapat mengecek kondisi program, nilai variabel, maupun alur proses program secara lebih detail.

Adapun fitur-fitur tool debugger diantaranya:

1. Fitur Step Into digunakan untuk menjalankan program baris per baris dan masuk ke dalam fungsi atau method yang sedang dipanggil agar proses di dalamnya dapat

diperiksa secara detail.

2. Fitur Step Over digunakan untuk menjalankan baris kode berikutnya tanpa masuk ke dalam fungsi atau method yang dipanggil, sehingga debugger hanya melewati proses tersebut dan melanjutkan ke baris selanjutnya.
3. Fitur Step Out digunakan untuk keluar dari fungsi atau method yang sedang diperiksa dan melanjutkan proses debug ke bagian setelah fungsi tersebut selesai dijalankan.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat:

<https://github.com/AnggraeniDwiZahra/Pemrograman-Mobile>

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Application class adalah sebuah kelas yang di instansiasi secara otomatis oleh android sistem ketika sebuah aplikasi diluncurkan. Yang juga bertanggung jawab untuk mengelola siklus hidup aplikasi secara keseluruhan. Objek ini diinstansiasi sebelum Activity, Service, atau BroadcastReceiver dibuat, dan tetap aktif hingga aplikasi dihentikan atau dihapus dari memori.

Sehingga dapat dikatakan bahwa application class berfungsi sebagai kelas dasar untuk memelihara status aplikasi global dan melakukan tugas inisialisasi yang perlu dijalankan sebelum komponen aplikasi apa pun dibuat. Selain itu, kelas ini juga sering digunakan untuk konfigurasi awal aplikasi, seperti inisialisasi database, library, atau penyimpanan data yang dapat diakses oleh seluruh komponen aplikasi.